

Agentes: De ReAct manual a LangChaint

FCEFyN - UNC - Cátedra de Inteligencia Artificial

- Autor/es: Ing. Sandro Mauricio Comerci (LIDeSIA - FCEFyN)
- Directora: Dra. Laura Cecilia Diaz Davila (LIDeSIA - FCEFyN)

En la notebook anterior construimos un agente ReAct desde cero.

Implementamos manualmente:

- definición de tools,
- prompt ReAct,
- parser de acciones,
- ejecución de herramientas,
- observaciones,
- y bucle iterativo del agente.

En esta notebook vamos a subir un nivel de abstracción.

La idea principal es ver cómo frameworks como **LangChain** nos permiten construir agentes con menos código.

1. ¿Qué problema resolvemos ahora?

Nuestro agente ReAct manual funciona, pero tiene varias partes que tuvimos que programar nosotros:

```
LLM
 ↓
Prompt manual
 ↓
```

```
Parser con regex
 ↓
Tool registry manual
 ↓
Loop ReAct manual
 ↓
Observation manual
```

Cuando el agente crece aparecen problemas:

- mucho código repetitivo,
- difícil mantenimiento,
- manejo manual del estado,
- dificultad para agregar muchas tools.

LangChain aparece para abstraer gran parte de esa lógica.

2. ¿Qué es LangChain?

LangChain es un framework para construir aplicaciones basadas en modelos de lenguaje.

Permite trabajar con:

- modelos de chat,
- prompts,
- tools,
- agentes,
- cadenas,
- memoria,
- parsers,
- RAG,
- y workflows.

Lo usamos como una abstracción de lo que ya construimos manualmente.

La idea es:

```
ReAct manual
     ↓
LangChain
```

LangChain no reemplaza el concepto de ReAct.
LangChain puede implementar agentes ReAct automáticamente.

3. Instalación de dependencias

Instalamos LangChain y las integraciones necesarias.

También usamos `ddgs` para una tool simple de búsqueda web.

```
!pip install -q langchain langchain-openai langchain-community ddgs
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 127.9/127.9 kB 2.7 MB/s eta 0:00:
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.4/2.4 MB 34.8 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47.6/47.6 kB 3.1 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 36.4 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.6/6.6 MB 60.2 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.4/69.4 kB 4.4 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 73.1/73.1 kB 4.3 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.34.2 wh
```

```
import os
from getpass import getpass
from datetime import datetime
from typing import Optional

from ddgs import DDGS

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
```

5. Conexión con el modelo

En la notebook anterior ya vimos que el modelo puede estar alojado en un proveedor externo.

Aquí volvemos a usar **Hugging Face Router**, pero ahora desde LangChain.

La idea es:

```
LangChain
   ↓
ChatOpenAI
   ↓
Hugging Face Router
   ↓
Modelo remoto
```

```
from openai import OpenAI

if "HF_TOKEN" not in os.environ:
```

```
    os.environ["HF_TOKEN"] = getpass("Ingresá tu HF_TOKEN: ")

client = OpenAI(
    base_url="https://router.huggingface.co/v1",
    api_key=os.environ["HF_TOKEN"],
)

print("Cliente inicializado correctamente.")
```

```
Cliente inicializado correctamente.
```

```
models = client.models.list()

print("Modelos disponibles:")
for m in models.data[:30]:
    print("-", m.id)
```

```
Modelos disponibles:
- Qwen/Qwen3.8-27B
- deepseek-ai/DeepSeek-V4.1-Flash
- zai-org/GLM-5.3
- zai-org/GLM-5.3-Flash
- meta-llama/Llama-3.1-8B-Instruct
- XiaomiMiMo/MiMo-V2.6-Pro-RL
- moonshotai/Kimi-K3
- google/gemma-4-31B-it
- Qwen/Qwen3.6-35B-A3B
- Qwen/Qwen3.5-9B
- google/gemma-4-26B-A4B-it
- openai/gpt-oss-20b
- openai/gpt-oss-120b
- meta-models/Muse-Glimmer-30B
- XiaomiMiMo/MiMo-V2.6-Flash-RL
- deepseek-ai/DeepSeek-V4-Pro
- Qwen/Qwen3.8-2.4T-A95B
- meta-llama/Llama-3.3-70B-Instruct
- deepseek-ai/DeepSeek-R1
- thinkingmachines/Inkling
- zai-org/GLM-5.2
- deepseek-ai/DeepSeek-V4-Flash-0731
- google/gemma-3-4b-it
- Qwen/Qwen3-8B
- MiniMaxAI/MiniMax-M3
- deepseek-ai/DeepSeek-V4-Flash
- meta-llama/Llama-4-Scout-17B-16E-Instruct
- inclusionAI/Ling-3.0-flash
- Qwen/Qwen2.5-Coder-32B-Instruct
- ibm-granite/granite-4.2-8b
```

```
llm = ChatOpenAI(
    api_key=os.environ["HF_TOKEN"],
    base_url="https://router.huggingface.co/v1",
    model="Qwen/Qwen2.5-7B-Instruct",
    temperature=0.1,
)
```

6. Primera llamada al modelo con LangChain

Antes de construir un agente, probamos que el modelo responda correctamente.

```
response = llm.invoke("Explica en una frase qué es LangChain.")
print(response.content)
```

```
LangChain es una plataforma que conecta modelos de lenguaje de forma secuenci
```

7. De tools manuales a tools con LangChain

Anteriormente definíamos herramientas manualmente usando una clase `Tool`.

Ahora LangChain permite definir tools con un decorador:

```
@tool
def mi_tool(...):
    ...
```

La descripción de la función es importante porque el modelo la usa para decidir cuándo utilizarla.

```
@tool
def now_tool(query: str = "") -> str:
    """
    Devuelve la fecha y hora actual.
    Usar solo cuando el usuario pregunte por fecha, hora o momento actua
    """
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
@tool
def web_search(query: str) -> str:
    """
    Busca información en internet.
    Usar para preguntas que requieren información externa, conceptos, hi
    Input esperado: una consulta de búsqueda en texto.
    """
    with DDGS() as ddgs:
        results = list(ddgs.text(query, max_results=5))

    if not results:
        return "No se encontraron resultados."

    output = []

    for r in results:
        title = r.get("title", "Sin título")
        url = r.get("href", "Sin URL")
```

```
        body = r.get("body", "Sin resumen")

        output.append(
            f"Título: {title}\n"
            f"URL: {url}\n"
            f"Resumen: {body}"
        )

    return "\n\n".join(output)
```

```
tools = [now_tool, web_search]

for t in tools:
    print("Tool:", t.name)
    print("Descripción:", t.description)
    print()
```

```
Tool: now_tool
Descripción: Devuelve la fecha y hora actual.
Usar solo cuando el usuario pregunte por fecha, hora o momento actual.

Tool: web_search
Descripción: Busca información en internet.
Usar para preguntas que requieren información externa, conceptos, historia, n
Input esperado: una consulta de búsqueda en texto.
```

8. Tool calling directo

Antes de construir el agente, podemos ejecutar una tool directamente.

Esto nos permite ver que una tool sigue siendo una función, pero ahora está envuelta en una estructura compatible con LangChain.

```
print(web_search.invoke("qué es un agente ReAct"))
```

```
Título: ¿Qué es un agente ReAct? | IBM
URL: https://www.ibm.com/es-es/think/topics/react-agent
Resumen: ¿Qué es un agente ReAct? Un agente ReAct es un agente de IA que util

Título: ReAct Agent: Guía para entender sus funcionalidades y ...
URL: https://www.plainconcepts.com/es/react-agent/
Resumen: En este artículo exploraremos al detalle en qué consisten los ReAct

Título: Patrón de Agente ReAct - Tutorial Agentes IA y RAG
URL: https://www.frontendtechlead.com/es/learn-ai-agents/react-agent-pattern
Resumen: Introducido por Yao et al. en su artículo de 2022 "ReAct: Synergizin

Título: Patrón ReAct Agent: Arquitectura de agentes fiable
URL: https://www.agentpatterns.tech/es/agent-patterns/react-agent
Resumen: Domina el patrón ReAct en arquitecturas de agentes ReAct: bucle reas

Título: Cómo Funcionan los Agentes de IA: Arquitectura y Ciclo ReAct
```

```
URL: https://gurusup.com/es/blog/how-ai-agents-work
Resumen: Apr 9, 2026 · Un agente de IA funciona mediante un bucle iterativo e
```

```
print(web_search.name)
print(web_search.description)
```

```
web_search
Busca información en internet.
Usar para preguntas que requieren información externa, conceptos, historia, n
Input esperado: una consulta de búsqueda en texto.
```

9. Crear un agente con LangChain

Ahora usamos LangChain para construir un agente.

Ya no vamos a escribir manualmente:

- regex,
- parser,
- loop ReAct,
- scratchpad (espacio donde el agente guarda su razonamiento intermedio y el historial de uso de herramientas),
- ni observaciones.

LangChain se encarga de orquestar el ciclo entre el modelo y las tools.

En versiones recientes, LangChain recomienda usar `create_agent`, que construye un runtime de agente basado en LangGraph internamente.

```
from langchain.agents import create_agent
```

```
/usr/local/lib/python3.12/dist-packages/langgraph/checkpoint/serde/encrypted.
  from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
```

```
agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt=(
        "Sos un agente didáctico. "
        "Usá herramientas cuando sea necesario. "
        "Si la pregunta puede responderse sin herramientas, respondé dir
        "Si necesitás información externa, usá web_search. "
        "Si necesitás fecha u hora actual, usá now_tool."
    ),
)
```

10. Ejecutar el agente

Ahora invocamos el agente.

A diferencia del ReAct manual, ya no necesitamos parsear `Action:` ni agregar `Observation:` a mano.

LangChain maneja internamente la interacción entre:

```
modelo → tool → observación → modelo
```

```
result = agent.invoke(
    {"messages": [{"role": "user","content": "Explica brevemente qué es
)

result
```

```
{'messages': [HumanMessage(content='Explica brevemente qué es una
herramienta en un agente de IA.', additional_kwargs={}, response_metadata=
{}, id='c4e578e5-21f6-4a2b-847a-08f583d74cd7'),
  AIMessage(content='En el contexto de un agente de inteligencia artificial,
una herramienta es un recurso o función que el agente puede utilizar para
obtener información adicional o realizar tareas complejas que no están
dentro de su conjunto de conocimientos o capacidades predefinidas. Estas
herramientas pueden incluir búsquedas en internet, acceso a bases de datos,
cálculos matemáticos, entre otros. El agente utiliza estas herramientas
cuando la información requerida no está disponible en su base de
conocimientos interna o cuando necesita realizar una tarea específica que no
está programada en su lógica.', additional_kwargs={'refusal': None},
response_metadata={'token_usage': {'completion_tokens': 130,
'prompt_tokens': 377, 'total_tokens': 507, 'completion_tokens_details':
None, 'prompt_tokens_details': None, 'cached_tokens': 0}, 'model_provider':
'openai', 'model_name': 'Qwen/Qwen2.5-7B-Instruct-Turbo',
'system_fingerprint': None, 'id': 'oj4bof4-4YNCb4-9fba76885d8eb788',
'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019e26cf-509c-72a3-
87ee-c61dafd762f8-0', tool_calls=[], invalid_tool_calls=[], usage_metadata=
{'input_tokens': 377, 'output_tokens': 130, 'total_tokens': 507,
'input_token_details': {}, 'output_token_details': {}})]}
```

```
result["messages"][-1].content
```

```
'En el contexto de un agente de inteligencia artificial, una herramienta es
un recurso o función que el agente puede utilizar para obtener información a
dicional o realizar tareas complejas que no están dentro de su conjunto de c
onocimientos o capacidades predefinidas. Estas herramientas pueden incluir b
```

```
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Dame las 8 noticias del dia"
            }
        ]
    }
)
```

```
11
12 result["messages"][-1].content
```

```
'Aquí tienes 8 noticias del día basadas en las fuentes que he consultado:\n
\n1. **México**: El gobernador de Sinaloa se separa del cargo mientras la Fi
scalía de México investiga acusaciones de EE.UU. por narcotráfico. Rubén Roc
ha Moya se declara inocente de estas acusaciones.\n\n2. **México**: Cientos
de trabajadores mexicanos participan en una marcha multitudinaria por el Día
del Trabajo.\n\n3. **EE.UU.**: Un juez ordena al Gobierno de EE.UU. que devu
```

```
1 result = agent.invoke(
2     {
3         "messages": [
4             {
5                 "role": "user",
6                 "content": "¿Qué dia es hoy?"
7             }
8         ]
9     }
10 )
11
12 result["messages"][-1].content
```

```
'Hoy es 14 de mayo de 2026.'
```

13. Comparación: Agente manual vs LangChain

Parte	Agente manual	LangChain
Tools	Clase propia `Tool`	Decorador `@tool`
Prompt	Manual	`system_prompt` / templates
Parser	Regex manual	Interno
Loop	`while` manual	Runtime del agente
Observations	Agregadas manualmente	Gestionadas por el framework
Escalabilidad	Limitada	Más modular
Debugging	Más explícito	Más abstracto

14. ¿Qué es una skill?

Una **tool** suele ser una función individual.

Una **skill** puede pensarse como una capacidad más amplia y reutilizable del agente.

Por ejemplo:

```
Tool:
- web_search

Skill:
- investigación web
    - buscar
```

```
- resumir
- comparar fuentes
- extraer entidades
```

Una skill puede agrupar varias tools y una lógica de uso.

Esto permite diseñar agentes más modulares.

```
@tool
def research_skill(topic: str) -> str:
    """
    Skill simple de investigación.
    Internamente usa la tool web_search y luego devuelve un resumen comp
    """
    raw_results = web_search.invoke(topic)

    prompt = f"""
    Resumí la siguiente información en español, de forma breve y didácti

    Información:
    {raw_results}
    """

    response = llm.invoke(prompt)
    return response.content
```

```
research_skill("qué es LangChain")
```

```
'LangChain es un framework de código abierto que facilita la creación de apl
icaciones utilizando modelos de lenguaje de gran tamaño (LLMs). Este marco a
yuda a los desarrolladores a integrar inteligencia artificial de manera efic
iente, permitiendo la orquestación de código y la combinación de LLMs con fu
entes de datos externas. Las empresas lo utilizan para desarrollar aplicacio
nes avanzadas como chatbots y agentes virtuales, mejorando la interacción co
```

15. Convertir una skill en tool

Una skill también puede exponerse como una tool para que el agente la use.

Esto muestra una idea importante:

```
Las tools pueden ser simples funciones
o capacidades compuestas de más alto nivel.
```

```
@tool
def research_skill_tool(topic: str) -> str:
    """
    Investiga un tema en internet y devuelve un resumen breve en español
    Usar cuando el usuario pida investigar, resumir o explicar un tema e
    """
    return research_skill(topic)
```

```python
@tool
def research_skill(topic: str) -> str:
    """
    Investiga un tema en internet y devuelve un resumen breve en español

    Usar cuando el usuario pida investigar, resumir o explicar un tema q
    """
    raw_results = web_search.invoke(topic)

    prompt = f"""
    Resumí la siguiente información en español, de forma breve y didácti

    Información:
    {raw_results}
    """

    response = llm.invoke(prompt)
    return response.content
```

```python
tools_v2 = [now_tool, web_search, research_skill]
```

```python
research_skill.invoke("¿qué es MCP en el contexto de agentes de IA?")
```

```
'El Model Context Protocol (MCP) es un estándar abierto que facilita la inte
racción entre los agentes de inteligencia artificial (IA) y los sistemas, da
tos y herramientas externos. Este protocolo actúa como un puente, permitiend
o a los modelos de IA acceder directamente a archivos, APIs y herramientas s
in necesidad de procesos intermedios como embeddings o búsquedas vectoriale
s. El MCP resuelve el problema de integración N×M, que dificultaba la intero
```

```
Start coding or generate with AI.
```